



Arquitectura en la Nube

Semana 2 · Modelos de despliegue e identidad y acceso

Ing. Rodolfo Cañas Cervantes

Universidad de la Costa (CUC) · Ingeniería de Sistemas

Semanas 1 y 2 · Periodo 2026-2 · Barranquilla, Colombia



Modelos de despliegue de la nube

Cuatro formas de desplegar infraestructura — vista general antes de profundizar en cada una.

Nube pública

Infraestructura del proveedor, compartida (**multi-tenant**), pago por uso.

Nube privada

Dedicada a una sola organización: **control total**, a cambio de CapEx.

Nube híbrida

On-premise + pública conectadas: **lo sensible en casa, lo elástico afuera.**

Multi-nube

El mismo workload en 2+ proveedores, para reducir el **vendor lock-in.**

Ningún modelo es "**el correcto**": la decisión depende de regulación, datos sensibles, costos y estrategia de la organización.

Modelo de despliegue: Nube pública

Infraestructura del proveedor, multi-tenant, pago por uso — la base del modelo dominante.

Infraestructura del proveedor (compartida)



Aislamiento lógico entre tenants: cada uno ve solo sus recursos

Hardware del proveedor amortizado entre miles de clientes → **menor costo por unidad de cómputo**

Características

- Elasticidad casi ilimitada
- OpEx, sin inversión inicial
- AWS Academy es la plataforma de práctica del curso
- El cliente no ve ni gestiona el hardware físico

Topología de red en la nube

Región → VPC → subred pública + subred privada — patrón único en toda la industria.

REGIÓN

VPC / VNet — red virtual propia (ej. 10.0.0.0/16)

Subred pública (10.0.1.0/24)

Internet Gateway / NAT

Balanceador de carga

Tabla de rutas:
0.0.0.0/0 → Internet Gateway

Subred privada (10.0.2.0/24)

Servidor de aplicación

Base de datos

Sin salida directa a internet
(sale vía NAT si es necesario)

El patrón, no la marca

Toda nube usa el mismo modelo: **VPC** = red privada aislada, **subred** = porción del rango asociada a una zona, **tabla de rutas** = decide quién sale.

El nombre cambia según el proveedor; el mecanismo es el mismo.

Modelo de despliegue: Nube privada

Single-tenant, dedicada a una organización — control total a cambio de CapEx, sin elasticidad.

Datacenter dedicado — una sola organización (single-tenant)

Servidores propios
o arrendados

Virtualización privada
(vSphere · OpenStack · Hyper-V)

Red corporativa
acceso solo interno / VPN

Control total sobre hardware, datos y cumplimiento normativo

Ejemplos: banca con regulación estricta, gobierno, hospitales

El intercambio (trade-off)

- ✓ Control y seguridad física
- ✓ Cumplimiento / soberanía de datos
- ✗ CapEx alto y renovación de hardware
- ✗ Sin elasticidad real: la capacidad es la que se compró
- ✗ El equipo propio opera todo

Modelo de despliegue: Nube híbrida

Combina on-premise + pública conectadas — lo sensible en casa, lo elástico afuera.

On-premise / nube privada

Datos sensibles y sistemas legados
(core bancario, historia clínica, ERP)

Control físico y cumplimiento normativo

Capacidad fija (CapEx)

VPN / Direct
Connect /
ExpressRoute /
Interconnect

Nube pública (AWS)

Cargas elásticas y variables
(web, campañas, picos de demanda)

Bursting: desbordar picos hacia la nube

Pago por uso (OpEx)

Patrón típico: **los datos regulados nunca salen del datacenter propio**; la capa de presentación y los picos de tráfico viven en la nube pública.

Multi-nube: el riesgo de vendor lock-in

El mismo workload corre en 2+ proveedores — se diseña para reducir la dependencia de un solo proveedor.

Portabilidad se gana con disciplina: contenedores + IaC declarativa + datos en formatos abiertos

Tres niveles de lock-in — entre más abajo, más caro migrar:

1 · Infraestructura

Bajo lock-in: Terraform y contenedores son portables entre nubes — el código de IaC se reaplica con un provider distinto.

2 · Datos

Medio lock-in: formatos propietarios y costos de egreso (salida de datos) hacen cara una migración — Parquet y PostgreSQL viajan fácil, lo demás no.

3 · Aplicación

Alto lock-in: servicios gestionados propietarios (bases NoSQL, ML plataformas) amarran el código — migrar es reescribir.

Regla práctica: **la portabilidad se diseña desde el día 1** — una vez atado a un servicio propietario, salir cuesta 10× más que haber empezado portable.

Por qué AWS Academy es la plataforma de práctica

Decisión de diseño del curso: una sola consola para los laboratorios, sin dispersar el esfuerzo de aprendizaje.

Lo que AWS Academy ofrece

- **Cuenta gratuita** con créditos incluidos para cada estudiante
- Acceso a la consola real de AWS (misma que en producción)
- Laboratorios guiados y entornos abiertos para prácticas libres
- Cohorte del curso: 20 estudiantes con sandbox individual (ios-estNN)

Qué se transfiere a otras nubes

Toda la **mecánica** que se practica en AWS — VPC, subredes, IAM, S3, facturación por uso, alta disponibilidad — existe en cualquier nube con el mismo nombre conceptual y otro nombre comercial.

Lo que el estudiante gana con AWS Academy es **flujo de consola** y confianza operativa; el resto se mapea por analogía.

Por qué una sola plataforma: aprender una consola a fondo enseña los patrones. Pasarse entre tres consolas en paralelo confunde más de lo que aporta cuando se está empezando.

En **Integración de Soluciones** (otra asignatura) sí se trabaja OCI como cuarto proveedor — donde el caso de uso lo justifica.

Protección del acceso en la nube (multicloud)

IAM + MFA + mínimo privilegio — idéntico patrón en AWS IAM, Microsoft Entra ID y Google Cloud IAM.

Identidad (persona o servicio)



AWS IAM

- MFA · SSO
- Principio de mínimo privilegio
- Allow / Deny — Deny siempre gana
- Usuarios, roles y políticas JSON



Microsoft Entra ID

- MFA · SSO
- Principio de mínimo privilegio
- Allow / Deny — Deny siempre gana
- RBAC y Conditional Access



Google Cloud IAM

- MFA · SSO
- Principio de mínimo privilegio
- Allow / Deny — Deny siempre gana
- Permisos granulares por recurso

Los tres proveedores comparten el mismo patrón: **identidad** → **MFA** → **política** → **Allow/Deny**. Aprender uno es aprender los tres.

Autenticación vs. autorización

Primero se autentica quién eres (identidad) y luego se autoriza qué puedes hacer (políticas).

1 · AUTENTICACIÓN

¿Quién eres?

- Usuario + contraseña
- MFA (algo que tienes)
- SSO / federación
- Claves de acceso (servicios)

2 · AUTORIZACIÓN

¿Qué puedes hacer?

- Políticas y roles
- Permisos por recurso
- Allow / Deny explícito
- Mínimo privilegio

RECURSO

- Bucket S3
- VM / instancia
- Base de datos
- Función

Ejemplo cotidiano: entrar al edificio con carnet = autenticación; que tu carnet solo abra el piso 3 = autorización.
Autenticarse correctamente **NO implica tener permiso**: puedes estar dentro y aun así recibir "Access Denied".

Anatomía de una política de acceso

Una política de acceso es un documento JSON/YAML con cuatro elementos: efecto, principal, recurso y condición.

```
{
  "Version": "2012-10-17",
  "Statement": [{
    "Effect": "Allow",
    "Principal": {"AWS": "arn:aws:iam::123:user/ana"},
    "Action": ["s3:GetObject"],
    "Resource": "arn:aws:s3:::datos-cuc/*",
    "Condition": {"IpAddress": {"aws:SourceIp": "190.x.x.x"}}
  ]
}
```

Efecto (Effect)

Allow o Deny. Si hay un Deny explícito, **siempre gana** sobre cualquier Allow.

Principal

A quién aplica: usuario, rol o servicio (la identidad autenticada).

Recurso (Resource)

Sobre qué aplica: un ARN/identificador específico — mínimo privilegio = recursos exactos, no "*".

Condición (Condition)

Cuándo aplica: IP de origen, horario, MFA presente, etiquetas...

El esquema completo (4 elementos) se documenta como una **política JSON** en AWS — auditable, versionable, testeable.